

# Resource Frontiers for Forge

Exact Unbounded-Multiplicity Reasoning,  
Guarded Program Equivalence, and Population Synthesis

A concrete extension proposal with executable Python artifacts

15 September 2026

## Abstract

This article develops one coherent extension to the inspected Forge design: an exact reasoning lane for finitely many kinds of resources with unbounded natural multiplicities. A transition consumes and produces fixed multisets, and it is enabled only when its required resources are present. From finitely many upward-closed bad patterns, a backward antichain computation produces the entire minimal unsafe initial-state frontier. A two-sided certificate combines backward closure with an enabled run from every frontier point. Consequently, one checked object classifies *every* initial marking, rather than answering only one bounded search query.

An exact algebra of resource demands and returns supports proved repetition acceleration, compressed finite runs, equivalence of guarded straight-line programs, and positive self-covering witnesses. Pulling the unsafe frontier back through a nonnegative affine initialization yields least unsafe population thresholds and multiparameter Pareto frontiers; the finite enumeration bound is proved, not guessed. The package implements these algorithms with a separate, search-free checker. It contains 68 passing unit tests, 3,920 complete forward-oracle comparisons, and 423 replay records of explicitly different kinds. No Lean executable was available: this is not an implemented Lean tactic, a verified checker, or a measured improvement over another tactic. The article specifies the missing Lean integration down to semantic bridge obligations and proposed module responsibilities.

**Reading guide.** Sections 1–3 establish the extension and its scope. Sections 4–8 give algorithms and proofs. Sections 9–10 explain integration and implementation. Section 11 reports only executed experiments. Appendix A gives commands that run without installing search libraries.

## Contents

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>The extension, not another architecture</b>                            | <b>4</b>  |
| 1.1      | A bounded non-duplication claim . . . . .                                 | 4         |
| 1.2      | What the proposed lane returns . . . . .                                  | 4         |
| 1.3      | Why this is not a claim against <code>grind</code> . . . . .              | 5         |
| <b>2</b> | <b>A motivating example that changes with one guard</b>                   | <b>5</b>  |
| <b>3</b> | <b>The supported mathematical fragment</b>                                | <b>6</b>  |
| 3.1      | Explicit exclusions . . . . .                                             | 6         |
| <b>4</b> | <b>Exact summaries of guarded resource programs</b>                       | <b>7</b>  |
| 4.1      | What an execution summary must remember . . . . .                         | 7         |
| 4.2      | Repetition without expansion . . . . .                                    | 8         |
| 4.3      | A complete endpoint-equivalence procedure . . . . .                       | 8         |
| <b>5</b> | <b>Exact backward frontiers</b>                                           | <b>9</b>  |
| 5.1      | The predecessor of an upward pattern . . . . .                            | 9         |
| 5.2      | The worklist, including provenance deletion . . . . .                     | 9         |
| 5.3      | Why the unbounded algorithm terminates . . . . .                          | 10        |
| 5.4      | The two-sided certificate . . . . .                                       | 11        |
| 5.5      | A hand-checkable receipt for the mutex . . . . .                          | 12        |
| <b>6</b> | <b>Exact repetition acceleration without a new checker</b>                | <b>12</b> |
| 6.1      | The predecessor of a fixed loop star . . . . .                            | 12        |
| 6.2      | The implemented shortcut . . . . .                                        | 13        |
| 6.3      | Enabling guards survive the acceleration . . . . .                        | 13        |
| 6.4      | An unavoidable frontier-size obstruction . . . . .                        | 13        |
| <b>7</b> | <b>Initial-population and parameter synthesis</b>                         | <b>14</b> |
| 7.1      | Scalar populations with an exact first failure . . . . .                  | 14        |
| 7.2      | Several parameters: first substitute, then compress . . . . .             | 14        |
| 7.3      | A simple complete compiler and its exact cost . . . . .                   | 15        |
| 7.4      | Why fixed-initial-state pruning does not automatically transfer . . . . . | 16        |
| <b>8</b> | <b>Positive infinite-behavior certificates</b>                            | <b>16</b> |
| <b>9</b> | <b>A concrete Lean integration contract</b>                               | <b>16</b> |
| 9.1      | Separate the small proof library from the search . . . . .                | 17        |
| 9.2      | A source-level theorem needs a semantic bridge . . . . .                  | 17        |
| 9.3      | A first exact bridge: multisets over a finite place type . . . . .        | 18        |
| 9.4      | Where Leant and Djex help, specifically . . . . .                         | 18        |
| 9.5      | The actual proof returned to the user . . . . .                           | 19        |

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| 9.6       | Acceptance gates that do not substitute easier statements . . . . .  | 19        |
| <b>10</b> | <b>Implementation, formats, and resource costs</b>                   | <b>19</b> |
| 10.1      | The delivered package is deliberately small . . . . .                | 19        |
| 10.2      | Exact problem identity and parse validation . . . . .                | 20        |
| 10.3      | Certificate families and their authority . . . . .                   | 20        |
| 10.4      | A concrete frontier record . . . . .                                 | 20        |
| 10.5      | Cost model . . . . .                                                 | 21        |
| 10.6      | Libraries to reuse and what still needs an adapter . . . . .         | 21        |
| <b>11</b> | <b>Experiments and their limits</b>                                  | <b>22</b> |
| 11.1      | What actually ran . . . . .                                          | 22        |
| 11.2      | A complete forward oracle where completeness is justified . . . . .  | 22        |
| 11.3      | Differential arithmetic and parameter tests . . . . .                | 23        |
| 11.4      | A controlled acceleration ablation . . . . .                         | 23        |
| 11.5      | Corruption controls and genuine unknowns . . . . .                   | 24        |
| 11.6      | Runtime observations without a tactic benchmark . . . . .            | 24        |
| 11.7      | What these results do not establish . . . . .                        | 24        |
| <b>12</b> | <b>A deployment path with falsifiable priorities</b>                 | <b>25</b> |
| 12.1      | First deliver the finite checker theorem . . . . .                   | 25        |
| 12.2      | Make optimization preserve the mathematical contract . . . . .       | 25        |
| 12.3      | Evaluate where a general proof assistant actually benefits . . . . . | 25        |
| <b>13</b> | <b>Conclusion</b>                                                    | <b>26</b> |
| <b>A</b>  | <b>Rebuild and run the package</b>                                   | <b>26</b> |
| <b>B</b>  | <b>An evidence map for maintainers</b>                               | <b>27</b> |
| <b>C</b>  | <b>Boundary checklist</b>                                            | <b>28</b> |

# 1 The extension, not another architecture

## 1.1 A bounded non-duplication claim

The baseline is the Forge revision 58ea206bd7ad501b930add568151217bcc7f82a2. The review inspected the merged closure, analytic, quantitative, and conclusion material, the repository overview, and its neighboring-project analysis; it also inspected the current-scope portions of Leant and Djex. The merged design already includes invariant and ideal closure, finite-algebra covers, arithmetic witnesses, and pushdown summaries. Its existing trust architecture and proof-DAG idea are prerequisites, not contributions of this article [1, 2, 3, 4].

The review did not read every line of all twenty-seven archived submissions. Thus the precise claim is *new to the inspected merged design*, not a proof of absence from every historical file. Repository search was not usable as an absence test: even a known-present positive control returned no matches. The package preserves this limitation in `docs/REVIEW_SCOPE.md` rather than converting an empty search result into evidence.

The missing problem shape is *guarded, unbounded multiplicity*. Finitely many places do not imply finitely many states: a two-place system can reach  $(1, 0), (1, 1), (1, 2), \dots$ . Conversely, the target may be an inequality such as “at least two processes are critical”, not an equality observation propagated by polynomial substitution. These two facts make a different closure operation useful.

| Existing mechanism               | neighboring | What is reused                              | What this article adds                                                               |
|----------------------------------|-------------|---------------------------------------------|--------------------------------------------------------------------------------------|
| Finite-algebra covers            |             | Finite certificates for global behavior     | A frontier in $\mathbb{N}^d$ , not a finite enumeration of resource states           |
| Observable/ideal closure         |             | Target-directed closure as a proof strategy | Exact guarded predecessor thresholds and upward-set saturation                       |
| Pushdown proof DAGs              |             | Shared, compressed execution evidence       | Resource enabling domains and exact net effects, without stack semantics             |
| Integer witness/projection lanes |             | Exact arithmetic as a substrate             | A reusable all-initial-state reachability predicate before parameter synthesis       |
| Leant/Djex synthesis             |             | Checked adapter-term construction           | A semantic resource problem for the adapter to connect, not a new type-search engine |

This is an engineering and certificate-design contribution. Petri-net coverability, well-quasi-order termination, and composition of partial translations are not claimed as new mathematics. Backward coverability is a classical method; Geffroy, Leroux, and Sutre provide a concrete primary reference and a more sophisticated fixed-initial-state implementation [7]. Our deliberate choice is a small certificate language that supports a stronger *output contract*: the exact unsafe precondition for all initial markings.

## 1.2 What the proposed lane returns

The central result is a finite list  $B \subseteq \mathbb{N}^d$  with an equivalence

$$\text{Unsafe}(m) \iff \bigvee_{b \in B} \bigwedge_{i < d} b_i \leq m_i, \quad \text{for every } m \in \mathbb{N}^d. \quad (1)$$

The list is a minimal antichain, and each member carries an actual enabled path to a bad pattern. This is neither a guessed invariant nor a table of tested initial states.

Once (1) is available, the same result answers several different questions. A fixed marking is safe precisely when it avoids the upward closure of  $B$ . A family  $m(n) = an + c$  has a computable first unsafe population, or is safe for all populations. A family  $m(\theta) = A\theta + c$  has a computable minimal unsafe parameter frontier. A bad initial marking also has a finite execution witness, obtained by lifting a stored frontier path. Separately, the same exact path summaries decide guarded endpoint equivalence of compressed programs and check positive nontermination or unboundedness witnesses.

### 1.3 Why this is not a claim against grind

Lean’s current reference describes **grind** as a cooperating system of congruence closure, propagation, E-matching, case analysis, and arithmetic reasoning that constructs ordinary proof terms [8]. Nothing here replaces that infrastructure. The new worker would provide a finite theorem such as (1); ordinary Lean reasoning can then specialize it to the user’s hypotheses.

No experiment in this package compares the worker with **grind**, **Aesop**, **omega**, or any other Lean tactic. In particular, the mutex example below may have a short hand-supplied invariant proof. The proposed gain is automatically obtaining a complete initial-state classification in a supported fragment, not an assertion that an existing tactic could never prove an individual theorem.

## 2 A motivating example that changes with one guard

Consider three token counts:  $w$  waiting processes,  $p$  permits, and  $c$  critical processes. The intended transitions are

$$\begin{aligned} \text{enter} : (w, p, c) &\mapsto (w - 1, p - 1, c + 1), & w \geq 1, p \geq 1, \\ \text{leave} : (w, p, c) &\mapsto (w + 1, p + 1, c - 1), & c \geq 1. \end{aligned}$$

The bad pattern is  $c \geq 2$ . Coordinates are natural numbers, and the guards are part of the semantics. The executed prototype produces

$$B_{\text{mutex}} = \{(0, 0, 2), (1, 1, 1), (2, 2, 0)\}. \quad (2)$$

The three members correspond to already being bad, entering once, and entering twice. A configuration is unsafe exactly when it covers one of these vectors.

For initial state  $(n, 1, 0)$ , none of the three patterns is covered, for any  $n \in \mathbb{N}$ . Therefore the one-permit system is safe for every population. For initial state  $(n, k, 0)$ , unsafety is exactly

$$n \geq 2 \quad \text{and} \quad k \geq 2.$$

This is a parameter classification, not a check for populations up to a chosen cutoff.

Now delete only the permit consumption from **enter**. The changed transition is

$$\text{enter}_{\text{broken}} : (w, p, c) \mapsto (w - 1, p, c + 1), \quad w \geq 1.$$

The same algorithm returns

$$B_{\text{broken}} = \{(0, 0, 2), (1, 0, 1), (2, 0, 0)\}.$$

For  $(n, 1, 0)$  the least unsafe population is now  $n = 2$ . For  $(n, k, 0)$  the unique minimal unsafe parameter pair is  $(2, 0)$ . The certificate supplies the two-entry execution; it does not merely report a solver model whose transition ordering is unknown.

**Example 2.1** (An infinite reachable set with a one-vector safety frontier). A transition consumes  $(1, 0)$  and produces  $(1, 1)$ . From  $(1, 0)$  it generates arbitrarily many tokens in the second place. Nevertheless, the bad pattern  $(2, 0)$  is never covered. The complete unsafe initial-state frontier is just  $\{(2, 0)\}$ . A finite-state exploration cannot exhaust this particular reachable set, but the backward frontier closes immediately. This distinguishes infinite multiplicity from the finite-algebra mechanism already in Forge.

**Example 2.2** (Coverability is not exact reachability). With one transition  $x \mapsto x + 2$  from  $x = 0$ , the target pattern  $x \geq 1$  is coverable, while the exact marking  $x = 1$  is not reachable. The worker accepts the former query and makes no claim about the latter. A source adapter must not translate equality to upward coverage.

### 3 The supported mathematical fragment

**Definition 3.1** (Ordinary resource net). Fix  $d \in \mathbb{N}$ . A marking is  $m \in \mathbb{N}^d$ . A transition  $t$  has consumption  $c_t \in \mathbb{N}^d$  and production  $p_t \in \mathbb{N}^d$ . Its step relation is

$$m \xrightarrow{t} m' \iff c_t \leq m \text{ and } m' = m - c_t + p_t, \quad (3)$$

where order is coordinatewise. There are finitely many transitions. The relation  $\rightarrow^*$  permits any finite sequence of enabled transitions, including the empty sequence.

Natural subtraction in (3) is harmless only because the enabling guard is retained. The input parser accepts literal nonnegative integers, not floating-point numbers or Boolean encodings of integers. This is the standard ordinary place/transition-net fragment [7]; no Petri-net package is needed to define it.

For a finite set  $T \subseteq \mathbb{N}^d$  of bad patterns, write

$$\uparrow T = \{m : \exists b \in T, b \leq m\}, \quad \text{Unsafe}(m) \iff \exists z \in \uparrow T, m \rightarrow^* z.$$

Safety means that *every finite execution* avoids the bad set:

$$\text{Safe}(m) \iff \neg \text{Unsafe}(m).$$

This is a universal safety property under nondeterministic scheduling. It says nothing by itself about fairness, eventual progress, deadlock freedom, or termination.

**Lemma 3.2** (Frame monotonicity). *If  $m \xrightarrow{t} m'$  and  $z \in \mathbb{N}^d$ , then  $m + z \xrightarrow{t} m' + z$ . The same implication holds for any fixed finite transition word.*

*Proof.* The guard  $c_t \leq m$  implies  $c_t \leq m + z$ . Since the subtraction is enabled,  $(m + z) - c_t + p_t = (m - c_t + p_t) + z$ . Induct on word length.  $\square$

Thus Unsafe is upward-closed. Extra tokens cannot disable an ordinary transition. This is the structural fact that makes the frontier method possible.

#### 3.1 Explicit exclusions

Zero tests and inhibitor arcs can make extra tokens disable a transition. Resets do not have the constant-effect form (3). Negative initial-family coefficients destroy parameter monotonicity. Ordered queues are not multisets. Unbounded data values require more than a fixed set of places. The implemented schema refuses these extensions rather than guessing a translation. Some admit other correct algorithms, but not automatically the formulas proved here.

Finite control can be encoded by one-hot control places, provided the source bridge proves the required control-state invariant. This does not justify silently interpreting every vector of control tokens as a source state. The object net has semantics on all its markings; the source adapter determines which of them represent source configurations.

The dimension  $d$  is fixed for a query. A theorem quantified over the number of tokens is supported; a theorem quantified over an arbitrary number of distinct place names or transition schemas is a different problem. The prototype supports the degenerate zero-dimensional frontier, but its multiparameter matrix encoding requires  $d > 0$  so that the number of columns is unambiguous.

## 4 Exact summaries of guarded resource programs

### 4.1 What an execution summary must remember

A list of transition counts gives a net effect, but not an enabling condition. A program that spends a token and then earns it back has zero effect and requires one token. Earning first and spending afterwards also has zero effect but requires none. Any equivalence or witness checker that discards this distinction is unsound.

For a finite word  $u$ , let its summary be  $(r_u, q_u) \in \mathbb{N}^d \times \mathbb{N}^d$ , interpreted as the partial function

$$F_u(m) = m - r_u + q_u \quad \text{with domain } \{m : r_u \leq m\}. \quad (4)$$

Here  $r_u$  is the exact minimum initial resource demand, not merely a sufficient upper bound. The vector  $q_u$  is the final marking when starting at  $r_u$ . The empty word has summary  $(0, 0)$ , and a primitive transition has summary  $(c_t, p_t)$ .

**Theorem 4.1** (Exact composition). *If  $u$  has summary  $(r, q)$  and  $v$  has summary  $(s, t)$ , then  $uv$  has summary*

$$(r, q) \star (s, t) = (r + (s - q)^+, t + (q - s)^+), \quad (5)$$

where positive parts are coordinatewise. This summary describes exactly both enabled inputs and endpoints.

*Proof.* Both segments execute exactly when  $m \geq r$  and  $m - r + q \geq s$ . Coordinatewise, the least such  $m$  is  $r + \max(s - q, 0)$ . Starting there, the first segment finishes at  $\max(q, s)$ ; after the second segment the marking is  $t + \max(q - s, 0)$ . Extra starting resources pass through by Lemma 3.2, giving (4) on the whole domain.  $\square$

The producer uses (5). The independent checker instead stores  $(r, e)$  with signed effect  $e = q - r \in \mathbb{Z}^d$ . Its concatenation formula is

$$(r, e) \diamond (s, f) = (\max(r, s - e), e + f). \quad (6)$$

The implementation therefore does not call the producer's natural-pair assembler during replay. The two programs still share a mathematical specification and the Python runtime; different formulas are additional testing leverage, not formal independence.

**Corollary 4.2** (Associativity and uniqueness). *The operation  $\star$  is associative with identity  $(0, 0)$ . Two summaries denote the same partial function if and only if their demand and return vectors are equal.*

*Proof.* The domain of (4) has unique least element  $r$ , so the partial function determines  $r$ , and its value at  $r$  determines  $q$ . Associativity follows from associative composition of partial functions and uniqueness of summaries. The empty word is the identity.  $\square$

This argument avoids a long case split over componentwise maxima. The algebra is an exact normal form for this program fragment, not a proposed new algebraic structure in general mathematics.

## 4.2 Repetition without expansion

**Theorem 4.3** (Power summary). *Let a program have summary  $(r, q)$  and signed effect  $e = q - r$ . For  $n = 0$  its zeroth power has summary  $(0, 0)$ . For  $n \geq 1$ ,*

$$r_n = r + (n - 1)(-e)^+, \quad e_n = ne, \quad (7)$$

$$q_n = q + (n - 1)(e)^+. \quad (8)$$

*These are exact domain and endpoint summaries of  $n$  repetitions.*

*Proof.* Before copy  $j$ , counting from  $j = 0$ , the marking is  $m + je$ . Every copy is enabled if and only if  $m + je \geq r$  for  $0 \leq j < n$ . For a nonnegative coordinate of  $e$  the strongest requirement is  $m_i \geq r_i$ ; for a negative coordinate it is  $m_i \geq r_i - (n - 1)e_i$ . This proves the demand formula. The net effect is  $ne$ , and adding it to  $r_n$  gives (8). The empty repetition is treated separately, so no  $n - 1$  expression is used outside its intended branch.  $\square$

The compressed-program language has four operations:

```
empty
step(transition_index)
seq(left_node, right_node)
repeat(body_node, natural_count)
```

Nodes are stored in topological order, and references must point strictly backwards. The checker derives a summary and a primitive-transition count for each node. Cycles, future references, unknown operations, and excessive intermediate integer sizes are rejected or refused under explicit resource limits. It does not flatten the represented word.

**Example 4.4** (A  $10^{100}$ -step certificate). Let a step consume  $(2, 0)$  and produce  $(1, 1)$ . Its effect is  $(-1, 1)$ . For  $N = 10^{100}$ , the power summary has demand  $(N + 1, 0)$  and return  $(1, N)$ . The packaged three-node DAG—an empty node, a primitive step, and a repetition—certifies

$$(N + 1, 0) \xrightarrow{N \text{ steps}} (1, N).$$

It represents those steps; the experiment did not execute them. Starting instead at  $(N, 0)$  would leave a nonnegative formal endpoint  $(0, N)$ , but the last transition is disabled. The checker rejects this mutation because endpoint arithmetic is not an enabling proof.

## 4.3 A complete endpoint-equivalence procedure

Take two original compressed programs over the same object net. Recompute both summaries from those original program DAGs. Equality of  $(r, e)$  decides equality of the partial endpoint functions for every starting marking. Primitive step counts are deliberately not compared: doing nothing and executing an everywhere-enabled identity step twice have the same endpoint function.

If summaries differ, the algorithm constructs a separating initial marking. If  $r \neq s$ , at least one of  $r \not\geq s$  or  $s \not\geq r$  holds. Choose the corresponding minimum demand as the input: one program is enabled there and the other is not. If demands agree but effects differ, the common demand enables both and gives different endpoints. The negative certificate is checked by computing these two observations at the supplied marking.

**Theorem 4.5** (Guarded endpoint equivalence is decided). *For the compressed-program fragment above, exact summary comparison terminates and returns either an equivalence valid at every marking or a concrete separating marking. The claim observes enabledness and final marking only.*

*Proof.* Every finite DAG has finitely many nodes; each summary computation is finite integer arithmetic. Summary uniqueness proves the positive branch, and the construction in the preceding paragraph proves the negative branch.  $\square$

This theorem is not trace equivalence, intermediate-state safety equivalence, equal running time, or scheduler equivalence. A source optimizer may use it only when endpoint semantics is the required observation. Intermediate resource shortages *are* observed, because they alter the domain; intermediate visits to a bad marking are not automatically observed.

## 5 Exact backward frontiers

### 5.1 The predecessor of an upward pattern

**Lemma 5.1** (One-step inverse threshold). *For a transition with consumption  $c$  and production  $p$ , define*

$$\text{pre}_t(b) = c + (b - p)^+ . \quad (9)$$

*Then*

$$\{m : \exists m' \geq b, m \xrightarrow{t} m'\} = \uparrow\{\text{pre}_t(b)\}.$$

*Moreover, firing  $t$  from  $\text{pre}_t(b)$  yields  $\max(b, p) \geq b$ .*

*Proof.* The left-hand condition is  $m \geq c$  and  $m - c + p \geq b$ . For each coordinate, if  $b_i \leq p_i$ , the enabling requirement  $m_i \geq c_i$  suffices. If  $b_i > p_i$ , the two inequalities reduce to  $m_i \geq c_i + b_i - p_i$ . These are exactly (9). At the threshold, the endpoint is  $(b - p)^+ + p = \max(b, p)$ .  $\square$

This standard cover-predecessor construction appears in the primary coverability literature [7]. We include a complete proof because the guard and direction of the inequality are the central obligations of the proposed Lean checker.

The formula is monotone in  $b$ . Thus the predecessor of a dominated basis member contributes nothing that cannot eventually be covered by the predecessor of a smaller member. This permits antichain minimization without retaining every discovered marking in the active search frontier.

### 5.2 The worklist, including provenance deletion

The producer maintains an immutable history of entries  $(b, u, j)$ , where  $u$  is an enabled compressed run from  $b$  to a marking covering original target  $T_j$ . A separate set of active identifiers describes the current antichain; a FIFO queue holds pending active or stale identifiers.

```

seed each original target b with its empty witness
while pending identifiers remain:
  take an identifier i
  if i is no longer active: continue
  for each original transition t:
    candidate := consume[t] + positive(b[i] - produce[t])
    if some active a ≤ candidate: continue
    witness := seq(step(t), witness[i])
    append (candidate, witness, target[i]) to immutable history
    remove all active a with candidate ≤ a
    activate and enqueue the new identifier
return surviving basis and local closure witnesses

```

The prefixing order in `seq(step(t), witness[i])` is essential. The new predecessor executes  $t$  first, reaches a marking above  $b_i$ , then executes the previously stored suffix by frame monotonicity. Reversing the order can keep a plausible net effect while making the run invalid.

Deleting an active entry does not delete or recycle its provenance. Its run may remain a child of another run node. Only after the search finishes does a reachability traversal compact the DAG, renaming references consistently. This separates semantic dominance in the frontier from memory reachability in the proof graph.

Operational limits count candidate expansions and admitted history entries. Exhaustion returns UNKNOWN with statistics and no exact-frontier certificate. An incomplete worklist can contain useful positive witnesses, but it cannot establish the missing backward-closure obligations. The public frontier API therefore does not call a partially explored set “safe”.

### 5.3 Why the unbounded algorithm terminates

**Lemma 5.2** (Dickson’s property for a fixed number of counters). *Every infinite sequence in  $\mathbb{N}^d$  contains indices  $i < j$  with  $x_i \leq x_j$  coordinatewise. In particular, every antichain in  $\mathbb{N}^d$  is finite.*

*Proof.* Every infinite sequence of natural numbers has an infinite nondecreasing subsequence. If a value occurs infinitely often, take the constant subsequence. Otherwise each finite set of values occurs only finitely often, so successively choose strictly larger values from later positions. Apply this fact to the first coordinate of a sequence in  $\mathbb{N}^d$ , then to the second coordinate of the retained subsequence, and so on. After finitely many restrictions, all  $d$  coordinates are nondecreasing. Any two distinct positions in the final subsequence give the desired pair. The case  $d = 0$  is immediate.  $\square$

**Theorem 5.3** (Termination and completeness of saturation). *Without operational cutoffs, the worklist computation terminates and computes the minimal finite basis of Unsafe.*

*Proof.* Each admitted candidate is outside the upward closure of the current active antichain. The represented upward set only grows: deleting a dominated old member does not remove any represented marking because the newly admitted smaller member covers its whole upward cone. Suppose infinitely many candidates were admitted. By Lemma 5.2, an earlier candidate would be componentwise below a later one. Its upward cone is still represented when the later one is considered, contradicting that admission. Hence only finitely many entries are admitted. Each creates finitely many transition tasks, so the queue empties.

Every admitted entry has a genuine positive run by initialization, Lemma 5.1, and frame monotonicity. At termination every surviving entry has been expanded, and every predecessor is covered by the final active basis. If an older entry was skipped because it was dominated, monotonicity of (9) makes its predecessor obligations redundant. Original targets remain covered

throughout. The exactness theorem below now identifies the represented set with Unsafe. Since the active basis is an antichain, it is the minimal basis.  $\square$

The proof is qualitative. It is not a polynomial bound on the number or size of intermediate vectors. A terminating coverability method can still be impractical on difficult inputs. The implementation therefore exposes budget exhaustion even though the unbounded mathematical algorithm is complete.

## 5.4 The two-sided certificate

The checker is not asked to verify the saturation procedure. It receives a candidate finite list  $B$ , a compressed run  $u_b$  and original-target index for each  $b \in B$ , and finite membership witnesses for the following conditions:

$$\forall t_0 \in T, \quad \exists b \in B, \quad b \leq t_0, \quad (10)$$

$$\forall b \in B \quad \forall t, \quad \exists a \in B, \quad a \leq \text{pre}_t(b), \quad (11)$$

$$\forall b \in B, \quad u_b \text{ is enabled at } b \text{ and ends above some } t_0 \in T. \quad (12)$$

Membership witnesses are just indices into  $B$ . The checker recomputes every threshold and every run summary against the supplied original net. Sortedness and pairwise incomparability certify the additional minimal-antichain claim.

**Theorem 5.4** (Exact frontier certificate). *If (10)–(12) hold, then for every  $m \in \mathbb{N}^d$ ,*

$$\text{Unsafe}(m) \iff \exists b \in B, \quad b \leq m.$$

*If  $B$  is an antichain, its members are exactly the minimal unsafe markings.*

*Proof.* For the right-to-left implication, choose  $b \leq m$  and execute  $u_b$ . Condition (12) makes this run valid from  $b$ , and Lemma 3.2 lifts it to  $m$ , with an endpoint still covering an original target.

For the other implication, consider a finite bad-reaching execution and induct backwards along it. Its last marking is in  $\uparrow B$  by (10). If a successor covers some  $b \in B$  via transition  $t$ , Lemma 5.1 says that its predecessor covers  $\text{pre}_t(b)$ . Condition (11) supplies  $a \in B$  below that threshold. Therefore the predecessor also lies in  $\uparrow B$ . Induction reaches the initial marking.

For minimality, if an unsafe  $x \leq b \in B$  exists, exactness gives  $a \in B$  with  $a \leq x \leq b$ . Incomparability forces  $a = b$ , hence  $x = b$ . Conversely, every minimal unsafe marking covers some  $b \in B$  and therefore equals it.  $\square$

*Remark 5.5* (What each half buys). Conditions (10) and (11) alone prove that  $\uparrow B$  is an *over-approximation* of unsafe initial states. Its complement is safely usable as an invariant region, but a member of  $B$  need not be genuinely unsafe. Positive runs alone give an *under-approximation*: they exhibit unsafe states, but say nothing about unexplored initial states. Exact parameter thresholds require both halves. This is a specific obligation of the new output contract, not a new generic trust architecture.

**Corollary 5.6** (A quantifier-free weakest safety precondition). *An exact frontier gives*

$$\text{Safe}(m) \iff \bigwedge_{b \in B} \bigvee_{i < d} m_i < b_i. \quad (13)$$

*This is the largest set of initial markings from which every finite execution avoids the bad patterns. It is forward invariant under every enabled transition.*

*Proof.* Negate Theorem 5.4. For forward invariance, a successor outside the safe set would have a finite bad-reaching suffix, which the enabled prefix step would extend to a bad-reaching execution from the original marking.  $\square$

The formula can be handed back to **grind** or arithmetic reasoning once the exactness theorem has been proved in Lean. No reachability search is needed for later specializations to new initial conditions.

## 5.5 A hand-checkable receipt for the mutex

For the three vectors in (2), use respectively the empty word, **enter**, and **enter enter**. The six predecessor obligations are

| Basis vector $b$ | $\text{pre}_{\text{enter}}(b)$ | $\text{pre}_{\text{leave}}(b)$ |
|------------------|--------------------------------|--------------------------------|
| $(0, 0, 2)$      | $(1, 1, 1)$                    | $(0, 0, 3)$                    |
| $(1, 1, 1)$      | $(2, 2, 0)$                    | $(0, 0, 2)$                    |
| $(2, 2, 0)$      | $(3, 3, 0)$                    | $(1, 1, 1)$                    |

Every displayed predecessor is above a member of the three-vector basis. This table, the three enabled words, and target membership are the entire mathematical certificate. Neither a guess about maximum population nor a separate invariant-discovery oracle is needed.

## 6 Exact repetition acceleration without a new checker

### 6.1 The predecessor of a fixed loop star

The resource summary is more than a compact receipt for a path already found. It also gives a finite exact acceleration of repeated execution. Let a fixed nonempty macro  $u$  have demand  $r$  and signed effect  $e$ , and fix a bad threshold  $b$ . For  $n \geq 1$ , the minimum input from which  $u^n$  is enabled and ends above  $b$  is

$$a_n = \max \left( r + (n-1)(-e)^+, b - ne \right), \quad a_0 = b. \quad (14)$$

The maximum is coordinatewise in the integers; its first argument ensures a natural result.

**Theorem 6.1** (A proved finite loop-star bound). *Define*

$$H = \max \left( \{1\} \cup \left\{ \left\lceil \frac{(b_i - r_i)^+}{e_i} \right\rceil : e_i > 0 \right\} \right).$$

*Then the exact predecessor set of the bad cone under any number of copies of  $u$  is*

$$\bigcup_{n \geq 0} \text{Pre}_{u^n}(\uparrow\{b\}) = \uparrow \text{Min}\{a_0, a_1, \dots, a_H\}.$$

*If  $e \geq 0$ , only two candidate generators are needed:  $b$  and  $a_H$ .*

*Proof.* Equation (14) follows by combining the power demand with the endpoint condition  $m + ne \geq b$ . In a positive-effect coordinate,  $a_{n,i} = \max(r_i, b_i - ne_i)$  becomes the constant  $r_i$  by  $n = H$ . A zero-effect coordinate is constant for all  $n \geq 1$ . In a negative-effect coordinate, both arguments of the maximum are nondecreasing in  $n$ . Consequently  $a_H \leq a_n$  for all  $n > H$ , so later cones add nothing.

When  $e \geq 0$ , every coordinate of  $a_n$  is nonincreasing for  $n \geq 1$ . Thus  $a_H \leq a_n$  for all  $1 \leq n \leq H$  as well. The zero-copy case remains  $b$  and cannot generally be discarded.  $\square$

## 6.2 The implemented shortcut

**Status.** Productive *single transitions* are accelerated in the delivered producer. The general fixed-macro theorem above is proved here, but automatic multi-transition macro discovery and general mixed-effect star compilation are not implemented.

For a transition with  $p_t \geq c_t$ , set  $r = c_t$  and  $e = p_t - c_t$ . Its saturated predecessor is especially simple:

$$(a_H)_i = \begin{cases} r_i, & e_i > 0, \\ \max(r_i, b_i), & e_i = 0. \end{cases}$$

The producer computes  $H$ , proposes  $a_H$ , and records the positive prefix `repeat(step(t), H)`. It uses this candidate instead of the ordinary one-step predecessor. This is safe because  $a_H$  is genuinely unsafe whenever  $b$  is, and  $a_H \leq a_1$  ensures that the original one-step predecessor is still covered. The zero-copy case is already represented by the current  $b$ .

No new trusted rule or certificate schema is needed. The final checker still verifies backward closure against every *original primitive transition* and reconstructs the repeated positive witness from original consumption/production vectors. An incorrect accelerator can therefore cause failure or waste time, but it cannot pass solely because the search calls a vector a “limit”.

For  $x \mapsto x + 1$  and bad threshold  $x \geq N$ , the plain predecessor search descends through  $N, N - 1, \dots, 0$ . The accelerated mode jumps from  $N$  to 0 and checks the final closure. The witness is a repetition count  $N$ , not an unrolled word. At  $N = 10^{100}$  the difference is structural, not a larger fuel allocation.

## 6.3 Enabling guards survive the acceleration

For a transition consuming  $(1, 0)$  and producing  $(1, 1)$  with bad target  $(0, N)$ , the exact frontier is

$$\{(0, N), (1, 0)\} \quad (N > 0).$$

The zero-copy target must remain: a state already holding  $N$  output tokens is bad even without the enabling resource. The saturated predecessor must retain one enabling token: a zero marking cannot start the loop merely because its effect is nonnegative. Both facts are tested.

A useful later composition is to let bounded lasso discovery propose a multi-transition word, recompute its exact global summary, and admit it as an accelerated macro when its overall effect is nonnegative. This avoids a fixed-initial-state scope leak: the discovery may use one initial state, but the summary theorem gives the word’s exact domain at *every* state. Only the word, not the discovery’s reachability assumptions, is reused. This integration remains a design proposal.

## 6.4 An unavoidable frontier-size obstruction

Acceleration cannot make every explicit antichain small. A transfer step consuming  $(1, 0)$  and producing  $(0, 1)$ , with bad pattern  $(0, N)$ , has exact unsafe basis

$$\{(0, N), (1, N - 1), \dots, (N, 0)\}.$$

All  $N + 1$  vectors are incomparable. Any certificate format that explicitly lists the minimal unsafe basis must therefore contain at least  $N + 1$  vectors. The mixed-effect star theorem gives a finite bound but cannot defeat this output-size lower bound. A future arithmetic representation could compress the predicate to  $x + y \geq N$ ; the present antichain schema intentionally does not pretend to provide that extension.

## 7 Initial-population and parameter synthesis

### 7.1 Scalar populations with an exact first failure

Let

$$m(n) = an + c, \quad a, c \in \mathbb{N}^d, \quad n \in \mathbb{N}.$$

Fix an exact unsafe basis  $B$ . A member  $b$  is impossible for this family if some coordinate satisfies  $a_i = 0$  and  $c_i < b_i$ . Otherwise define

$$n_b = \max \left( \{0\} \cup \left\{ \left\lceil \frac{b_i - c_i}{a_i} \right\rceil : a_i > 0 \right\} \right). \quad (15)$$

The implementation equivalently replaces the numerator by its positive part before nonnegative ceiling division.

**Theorem 7.1** (Exact population threshold). *If every  $b \in B$  is impossible, all members of the initialization family are safe. Otherwise, with  $n_* = \min n_b$  over feasible  $b$ ,*

$$\text{Unsafe}(m(n)) \iff n \geq n_*.$$

Thus  $n_*$  is the least unsafe population;  $n < n_*$  is the exact safe range.

*Proof.* For a feasible  $b$ , the conjunction  $an + c \geq b$  is equivalent to the lower bounds on  $n$  in (15). Infeasible  $b$  can never be covered. The union of the feasible upper rays is the upper ray beginning at their least threshold. Apply Theorem 5.4.  $\square$

The producer computes ceiling divisions. The checker uses a different characterization: a supplied  $n_b$  must cover  $b$ , and, if  $n_b > 0$ , its predecessor  $n_b - 1$  must not. Monotonicity then proves minimality. Infeasible basis members require an actual zero-slope obstructing coordinate. A null global threshold means *all safe*, but only after the exact frontier and all these obligations have checked. It is not the output of a failed search.

The scalar algorithm has  $O(|B|d)$  arithmetic operations, in addition to the cost of checking the embedded frontier. Integer bit complexity is separate from this operation count.

### 7.2 Several parameters: first substitute, then compress

For  $A \in \mathbb{N}^{d \times k}$  and  $c \in \mathbb{N}^d$ , initialize at

$$m(\theta) = A\theta + c, \quad \theta \in \mathbb{N}^k.$$

The already-proved exact unsafe predicate is

$$U(\theta) = \bigvee_{b \in B} \bigwedge_{i < d} \left( c_i + \sum_{j < k} A_{ij} \theta_j \geq b_i \right). \quad (16)$$

This formula is available even when an explicit Pareto-frontier search would be too large. It should be returned as the semantic result before attempting optional normal-form compression. The current command-line prototype requests the compressed certificate and returns UNKNOWN if its enumeration budget is insufficient; it does not yet emit a separately certified formula-only result.

A basis vector  $b$  is *eligible* if every row with  $c_i < b_i$  has at least one positive coefficient  $A_{ij}$ . Ineligible vectors cannot be covered by any parameter choice. Let  $B'$  be the eligible subset. Define

$$C_j = \max \left( \{0\} \cup \left\{ \left\lceil \frac{(b_i - c_i)^+}{A_{ij}} \right\rceil : b \in B', i < d, A_{ij} > 0 \right\} \right). \quad (17)$$

These caps are conservative coordinate bounds, not necessarily the smallest box containing the final minimal parameter frontier.

**Theorem 7.2** (Clipping preserves the entire unsafe predicate). *Let  $\bar{\theta}_j = \min(\theta_j, C_j)$ . Then*

$$U(\theta) \iff U(\bar{\theta}) \quad \text{for every } \theta \in \mathbb{N}^k. \quad (18)$$

*Every minimal unsafe parameter vector therefore lies in the finite box  $\prod_{j < k} \{0, 1, \dots, C_j\}$ .*

*Proof.* Since  $A$  has nonnegative entries and  $\bar{\theta} \leq \theta$ , the implication  $U(\bar{\theta}) \Rightarrow U(\theta)$  is immediate.

For the converse, choose  $b \in B'$  covered at  $\theta$  and consider a row  $i$ . If  $c_i \geq b_i$ , that row remains satisfied. Otherwise, if no positive-coefficient parameter in this row was clipped, its value is unchanged. If some  $j$  with  $A_{ij} > 0$  was clipped, then (17) gives

$$A_{ij}\bar{\theta}_j = A_{ij}C_j \geq b_i - c_i.$$

That one contribution suffices for the row after clipping. Every row remains satisfied, so the same  $b$  is covered at  $\bar{\theta}$ .

If a minimal unsafe  $\theta$  lay outside the box, clipping would give a strictly smaller unsafe vector, contradicting minimality.  $\square$

### 7.3 A simple complete compiler and its exact cost

Enumerate the proved box lexicographically. At each point, evaluate (16); retain only componentwise-minimal unsafe vectors. The certificate contains the resulting antichain  $F$ , an original basis witness for each  $f \in F$ , the caps, and a complete box table labeling each point either safe or dominated by an indexed  $f \in F$ .

The checker verifies cap adequacy by multiplication inequalities, rather than rerunning the producer's ceiling-division calculation. It verifies that each purported minimum is unsafe. It then checks every box label against the exact source formula. Antichain minimality plus full coverage and Theorem 7.2 prove

$$U(\theta) \iff \exists f \in F, f \leq \theta \quad (\theta \in \mathbb{N}^k).$$

This is not an extrapolation from tests in a box: the clipping theorem is the bridge beyond the box.

Let  $G = \prod_j (C_j + 1)$ . A direct evaluation costs  $O(G|B|dk)$  arithmetic operations, with additional antichain comparisons and  $G$  labels in the certificate. The prototype deliberately chooses a transparent complete method rather than pretending that a potentially enormous parameter space has become cheap. Its box budget is checked before enumeration. More compact arithmetic or rectangle-cover certificates can be added later without changing (16); none is claimed implemented here.

**Example 7.3** (A tradeoff frontier, not a single threshold). With no transitions, one place, bad pattern  $x \geq 5$ , and initialization  $x = 2a + 3b$ , the minimal unsafe parameter points are

$$F = \{(0, 2), (1, 1), (3, 0)\}.$$

The system is unsafe at a parameter exactly when it dominates one of these tradeoffs. Formula (17) gives  $C = (3, 2)$ , and all three minima lie in that proved box. A single number called “the cutoff” would discard the distinction between the two resources.

## 7.4 Why fixed-initial-state pruning does not automatically transfer

A coverability tool solving one query may derive an invariant from its particular initial marking and discard predecessors inconsistent with that invariant. That can be entirely correct for its original query. It cannot, without a new proof, supply the global equivalence (1).

For example, an invariant asserting one permit might justify safety from  $(n, 1, 0)$  while intentionally ignoring the genuinely unsafe states with two permits. Reporting the resulting pruned frontier as valid for all  $(n, k, 0)$  would erase the true threshold in  $k$ . The proposed integration must distinguish a *global frontier* from a *family-relative certificate*. Reusing an invariant-based tool is possible, but the invariant's initial-condition scope must survive translation and checking. The current prototype avoids this issue by computing the unpruned global basis.

## 8 Positive infinite-behavior certificates

**Status.** A bounded path search and a complete checker for the stated positive witness shape are implemented. No negative liveness or termination procedure is claimed.

A lasso consists of a finite prefix  $u$  taking the requested initial marking  $m_0$  to  $m$ , and a nonempty loop program  $v$  enabled at  $m$ . Let  $v$  have signed effect  $e$ .

**Theorem 8.1** (Self-covering lasso). *If  $e \geq 0$ , the prefix followed by infinitely many copies of  $v$  is an infinite enabled execution. If additionally  $e_j > 0$ , the count in place  $j$  is unbounded along that execution.*

*Proof.* After  $n$  copies the marking is  $m + ne \geq m$ . Frame monotonicity therefore makes the next copy enabled. Since  $v$  contains at least one primitive transition, repeating it yields infinitely many transitions rather than repeatedly denoting the empty word. If  $e_j > 0$ , the subsequence of loop-boundary markings has coordinate  $m_j + ne_j$ , which exceeds every natural bound for a sufficiently large  $n$ .  $\square$

The lasso search enumerates bounded paths and compares new markings with ancestors on the same path. A self-covering pair gives a prefix and loop. The checker discards the search history and derives the two summaries independently. A zero-effect loop certifies existential nontermination, not unboundedness. A loop increasing one place while consuming another does not qualify unless its full effect is nonnegative.

For the unbounded-output example, a one-step loop from  $(1, 0)$  has effect  $(0, 1)$ , proving unbounded output. For an identity step consuming and reproducing one token, the loop proves nontermination from one token but cannot prove unboundedness. For the depletion example from a fixed finite initial marking, no qualifying growth loop is found; the API returns UNKNOWN, not a termination theorem.

No fairness condition is assumed. A nondeterministic system may have this infinite execution and also have terminating executions. The conclusion is existential, and a source-level lasso requires a source path-lifting argument just as a finite counterexample does.

## 9 A concrete Lean integration contract

**Status.** Designed, not compiled or integrated. All names introduced in this section under `Forge.Resources` are proposed interfaces, not existing repository APIs.

## 9.1 Separate the small proof library from the search

The first Lean implementation should prove the finite certificate theorem, not formalize the search’s worklist or Dickson’s lemma. Theorem 5.4 needs only elementary step semantics, finite run correctness, backward closure, and induction on a finite execution. Termination of an untrusted producer is irrelevant to soundness of its successful output.

A concrete module plan is:

| Proposed module                   | Definitions and required results                                                                                                               |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Resources.Basic</code>      | Markings, consumption/production transitions, enabled steps, finite reachability; frame monotonicity and exact predecessor theorem             |
| <code>Resources.Summary</code>    | Acyclic compressed programs, demand/effect/length summaries; domain, endpoint, concatenation, power, and endpoint-equivalence theorems         |
| <code>Resources.Frontier</code>   | Finite Boolean certificate checker; one soundness theorem returning the exact unsafe equivalence and a corollary for safety                    |
| <code>Resources.Parameters</code> | Exact scalar thresholds, nonnegative affine initialization, clipping theorem, and finite Pareto coverage checker                               |
| <code>Resources.Reify</code>      | A narrowly supported multiset/count bridge to the exact object net; no silent erasure of guards or source predicates                           |
| <code>Resources.Tactic</code>     | Goal recognition, external candidate request, reconstruction, checker application, and optional specialization to the requested initialization |

The basic representation can be as simple as the following design fragment:

```
def Marking (d : Nat) := Fin d → Nat

structure Transition (d : Nat) where
  consume : Marking d
  produce : Marking d

-- The actual Step relation must include this conjunction:
-- (forall i, t.consume i ≤ m i) and
-- (forall i, m' i = m i - t.consume i + t.produce i)
```

For runtime storage, fixed-length arrays can be used with a proved interpretation into `Fin d → Nat`. Array lengths and node-reference bounds must be validated before any lookup; unchecked indexing with a default value would change the object problem.

The predecessor theorem reduces to coordinatewise natural arithmetic after splitting the order of  $b_i$  and  $p_i$ .  $\omega$  is an appropriate local tool for these linear residual goals, not a substitute for the reachability induction. Power-summary proofs additionally require algebraic and order lemmas about multiplication; they should not be advertised as solved by linear arithmetic alone.

## 9.2 A source-level theorem needs a semantic bridge

Suppose source states form a type  $S$ , with source transition relation  $R$ , an encoding  $E : S \rightarrow \mathbb{N}^d$ , and source bad predicate  $\text{Bad}$ . A forward simulation sufficient for safety is

$$R(s, s') \implies E(s) \rightarrow^* E(s'), \quad (19)$$

$$\text{Bad}(s) \implies E(s) \in \uparrow T. \quad (20)$$

Together with an initial encoding identity, object-level safety implies source-level safety. The abstract transition may correspond to multiple source steps, or a source step to an abstract finite path, as long as the proved relation states the direction actually used.

For a source counterexample, these implications are not enough. An abstract path can be spurious. One needs path lifting from represented states and target reflection, for example

$$E(s) \xrightarrow{t} m' \implies \exists s', R^*(s, s') \wedge E(s') = m', \quad (21)$$

$$E(s) \in \uparrow T \implies \text{Bad}(s). \quad (22)$$

These conditions are intentionally stronger than the safety bridge. For an infinite source execution, the lifting additionally needs a non-stuttering/progress argument: infinitely many abstract steps must not all be implemented by empty source paths. The proposed interface should give the safety and counterexample bridges different types so an over-approximation cannot accidentally acquire refutation authority.

**Example 9.1** (Why counting can lose a counterexample). A source system may distinguish processes by an identifier or maintain a relation between two resources, while the count map retains only the number of each kind. The count net can then enable a pairing that the source relation forbids. Proving that source steps preserve counts does not prove that every count step can be realized. Such a net can still yield useful safety proofs via (19), but its positive paths must be labeled abstract until a lifting theorem is available.

### 9.3 A first exact bridge: multisets over a finite place type

For source states that really are multisets of a finite place type, the bridge is direct. Enumerate the place type once, and map a multiset to its counts. A source transition consumes multiset  $C_t$  and produces  $P_t$ , with premise  $C_t \leq M$ , and updates to  $M - C_t + P_t$ .

Mathlib’s `Mathlib.Data.Multiset.Count` and `Mathlib.Data.Multiset.AddSub` expose the count/order and count/addition/subtraction lemmas needed for this representation, including `Multiset.le_iff_count`, `Multiset.count_add`, and `Multiset.count_sub` [9, 10]. The count arithmetic in subtraction is natural truncated subtraction. The enabling premise is therefore a semantic requirement, not an optional simplification hint.

The bridge proof should establish both directions for this exact multiset model. Count extensionality proves that the vector update reconstructs the intended multiset result. A finite enumeration with a bijection proof prevents missing or duplicated places. The typeclass dictionaries and enumeration choices used to obtain counts remain part of the source interpretation. The rolling documentation was inspected, but these imports and names were not compiled at Forge’s pinned revision; their integration is an acceptance task, not an executed result.

### 9.4 Where Leant and Djex help, specifically

Leant and Djex already support typed construction with checked reconstruction, including selected polymorphic and dictionary evidence in documented fragments [5, 6]. Forge’s own neighboring-project review already extracts their receipt and source-verification discipline [4]. Re-proposing those mechanisms would be duplication.

The useful task here is adapter construction. Given library theorems for counts of multiset updates and a particular source step constructor, request a term of the exact bridge proposition for that constructor. Type-directed synthesis can select and compose already-proved conversion lemmas, specialize them to the finite place enumeration, and fill routine proposition-plumbing gaps. It must not be asked to justify an erased guard or to infer behavioral equivalence merely because two expressions share a type.

Concretely, a resource adapter should supply four checked components: the state encoding, the initial-family equation, a forward step simulation, and the bad-predicate implication. Only an adapter with the additional lifting and reflection components can report source counterexamples

or source-level exact thresholds. The external resource search sees the finite integer problem, not Lean metavariables. Leant/Djex sees typed bridge obligations, not authority to assert that the search succeeded. No connection to either synthesizer was executed in this package.

## 9.5 The actual proof returned to the user

The desired final declaration is a specialized instance of a general certificate soundness theorem. In schematic form,

$$\text{checkFrontier}(N, T, C) = \text{true} \implies \forall m, \text{Unsafe}_{N,T}(m) \leftrightarrow \text{covers}(C.B, m).$$

For a source family, compose this with the exact bridge and the threshold theorem. Recompile should replay a finite object and a stable proof, not rerun saturation or depend on a solver service.

A generated proof must bind the certificate to the *actual elaborated source problem*. Comparing two copies of worker-supplied JSON does not establish that link. The neutral object problem must be constructed by the trusted Lean-side reifier or justified by a checked bridge theorem; the worker may only propose data for that fixed problem. The prototype’s exact structural binding checks exercise the object-level part of this contract, not the unimplemented Lean-side part.

For closed finite arithmetic, a reflection theorem may turn a proved Boolean checker equality into the semantic conclusion. The implementation must select an evaluation path consistent with Forge’s existing kernel/axiom policy; it must not silently substitute an unchecked external Boolean for a Lean proof. The  $10^{100}$  example is a useful acceptance test because an implementation that expands the word will be obviously wrong architecturally even when its mathematics is correct.

## 9.6 Acceptance gates that do not substitute easier statements

A first integration milestone is an exact multiset theorem for all populations in the mutex family, with a source-level counterexample at population two in the broken family. The threshold theorem must retain the original source guards and parameter quantifiers; a theorem about a separately defined vector toy does not close that milestone.

The next milestone proves a guarded endpoint equivalence between two original compressed programs, including a negative control with equal net effects and unequal enabling domains. The final milestone adds the parameter cap and lasso bridge, with source-progress obligations for infinite behavior.

Each milestone should record the toolchain and Mathlib revision, compile in a fresh process, inspect the final theorem’s axiom inventory, and reject query, guard, node-order, and parameter mutations. These are acceptance criteria for future implementation. No entry in this list is labeled passed in the supplied results.

# 10 Implementation, formats, and resource costs

## 10.1 The delivered package is deliberately small

`prototype/forge_resources/model.py` defines the producer’s net and natural-pair summaries. `producer.py` implements antichain saturation, scalar and Pareto compilation, bounded lasso search, and program comparison. `checker.py` imports neither file. It validates input shapes, reconstructs signed summaries, and checks finite obligations. `replay.py` is a fresh-process replay entrance, and `solve.py` is a usable command-line producer followed by the independent checker.

The production algorithms use Python integers, tuples, lists, dictionaries, `collections.deque`, and finite Cartesian products from `itertools`. No numerical tolerance, floating-point rank, symbolic algebra package, SAT library, or network access is involved. The main experiments and replay also run with `python -S`, which disables site packages.

## 10.2 Exact problem identity and parse validation

Each replay record has three fields: a diagnostic name, the expected original object problem/query, and a candidate certificate. The checker requires exact type-aware structural identity between the embedded query and the expected query. Python’s ordinary equality considers `True == 1`; therefore a dedicated recursive comparison also compares types. Two regression tests explicitly replace a query integer with a Boolean and require rejection.

The JSON decoder rejects duplicate object keys rather than silently keeping the last value. Unknown fields are rejected at the semantic boundaries: an extra `reset` or `inhibitor` field cannot be ignored as decorative metadata. Vectors have exact dimensions, transition names are unique, references are bounded, and all data claimed to be naturals reject negative numbers and Boolean values.

These are useful controls, not a claim of production-grade hostile-input hardening. In particular, a large valid antichain can still trigger expensive pairwise comparisons, and general parser nesting and process-level denial-of-service isolation need additional work. The checker should remain in a bounded worker even after a Lean port exists.

## 10.3 Certificate families and their authority

| Schema suffix               | Exact checked meaning                                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>frontier.v1</code>    | A minimal basis for all initial markings from which some bad pattern is coverable                                       |
| <code>threshold.v1</code>   | The least unsafe scalar parameter, or a proof that the whole stated nonnegative family is safe                          |
| <code>parameters.v1</code>  | The exact minimal unsafe parameter antichain for the stated affine family                                               |
| <code>run.v1</code>         | A supplied compressed finite program is enabled at the supplied start and has the claimed endpoint and primitive length |
| <code>lasso.v1</code>       | The supplied prefix and nonempty self-covering loop prove existential nontermination, or designated-place unboundedness |
| <code>equivalence.v1</code> | The two original programs have equal endpoint partial functions, or the supplied initial marking separates them         |

All schema strings have prefix `forge.resources..` A checked Python record is labeled `PYTHON_CHECKED`; no field calls it a Lean proof. Resource-limit exceptions are different from invalid-certificate exceptions. A bounded producer’s negative search outcome is `UNKNOWN`, not the negation of the query.

## 10.4 A concrete frontier record

The essential payload, omitting ordinary JSON punctuation, is

```

schema: forge.resources.frontier.v1
problem: exact original net, including targets
basis:
  - marking: natural vector
    run: root node of enabled positive witness
    target: index of original bad pattern
runs: topologically ordered empty/step/seq/repeat nodes
target_cover: one basis index for each original target
predecessor_cover:
  one row per basis element, one basis index per original transition

```

A closure witness is not a Boolean truth table: the checker recomputes the indicated comparison. A run node does not contain a trusted effect: the checker recomputes its effect. The target index points into the original bad patterns, not a worker-invented weaker goal.

For equivalence, the query itself contains both original compressed programs. The receipt contains only an equivalence verdict or a proposed separating initial marking, plus exact query binding. This prevents an unrelated pair of conveniently equal worker programs from replacing the user’s programs at the object boundary.

## 10.5 Cost model

Let  $d$  be the place count,  $t$  the transition count,  $b$  the final basis size, and  $\ell$  the reachable certificate-DAG node count. The prototype’s final frontier checker uses

$$O(b^2d + btd + \ell d)$$

arithmetic/comparison operations: pairwise antichain validation, indexed local closure checks, and run-summary reconstruction. Sorting validation has lower or comparable order in this simple representation. This bound excludes bit complexity and initial parsing.

The search uses linear scans for dominance. If  $M$  candidates are considered and the active basis never exceeds  $K$ , the dominance work is  $O(MKd)$ , with additional linear deletion scans. Producing indexed closure witnesses by searching the final basis can cost  $O(tb^2d)$ . These are bounds for the stated simple data structures, not claims about optimal coverability implementations.

Integer sizes can dominate all these counts. A short repetition node can carry a very large exponent, and nested summaries can increase bit sizes. The checker has separate bounds on input integer bits, intermediate-summary bits, number of nodes, dimension, transitions, basis elements, file size at the replay entrance, and parameter box cells. A limit refusal is an operational result and makes no mathematical negative claim.

## 10.6 Libraries to reuse and what still needs an adapter

The first Lean version needs the core finite-data and arithmetic facilities plus the two Mathlib multiset modules already identified. It does not need a Gröbner engine, linear programming solver, or numerical library. This is a useful difference from several existing Forge lanes: the whole prototype search and checker are standard-library-only, not merely the final replay.

For larger source workloads, SMPT is a concrete external candidate producer to investigate. Its official project documentation describes coverability-specific PDR, proof export/checking options, bounded counterexample search, and Tina-based simulation/enumeration [11]. These are real algorithmic resources, but their output is *not* automatically one of the certificates here. A fixed-initial-state invariant certificate needs a different semantic bridge from a global frontier, as Section 7.4 explains. PNML import, reductions, and tool-produced traces each need validation against the original finite net.

The practical reuse strategy is therefore asymmetric. Import a returned transition word and check its exact demand and endpoint immediately. Import a claimed invariant only through a separately implemented checker for that invariant language. Do not accept a bare solver verdict, and do not claim a global all-initial-state frontier from a fixed-query proof. No SMPT, Tina, or solver executable was installed or run for this article.

## 11 Experiments and their limits

### 11.1 What actually ran

The recorded environment used Python 3.13.5 on Linux x86-64. The main experiment identifier is `resource-frontiers-20260915-v1`; its seed is 20260915. A separate guarded-program suite uses seed 20260916. The latter is a seed, not a claim that the experiment occurred on a later date. These seeds have meaning only together with the supplied generators and are unrelated to similarly numbered experiments in Forge.

The package has 68 passing `unittest` tests and six replay schemas. A fresh `python -S` process checked all 423 stored records and reported that neither the producer nor its representation module was imported. Table 1 reports the records by meaning; these units are not summed with test assertions or oracle queries to manufacture a solved-goal count.

| Stored record type                           | Records |
|----------------------------------------------|---------|
| Exact unsafe initial-state frontier          | 94      |
| Scalar threshold / all-safe family           | 136     |
| Minimal unsafe parameter frontier            | 69      |
| Exact compressed finite run                  | 1       |
| Designated-place unbounded lasso             | 1       |
| Existential nontermination lasso             | 1       |
| Equal guarded endpoint partial functions     | 63      |
| Different guarded endpoint partial functions | 58      |
| Replay records, not Lean goals               | 423     |

Table 1: Stored replay objects, from `results/replay-log.txt`. Parameter records reuse frontier certificates; this is not a collection of 423 independent mathematical discoveries.

### 11.2 A complete forward oracle where completeness is justified

The main net corpus contains seven inspectable examples and sixty generated conservative nets: twenty with two places and forty with three. Each generated transition has equal total consumption and production. This provides a known invariant: every state reachable from an initial marking has the same total token count.

For each generated net, the oracle enumerates every initial vector of total at most six. From each such vector, a separately written forward breadth-first search exhausts the reachable state set. It continues even after finding a bad marking. The invariant bounds the reachable set to finitely many vectors, so this oracle has no hidden depth cutoff. It implements firing directly from the input dictionaries and does not call the predecessor or summary assembler.

The resulting 3,920 queries comprise 1,877 unsafe and 2,043 safe classifications. All agree with the backward frontier. Across these separate searches there are 16,649 visited-state occurrences; this is a sum over queries, not a distinct-state count. There were zero oracle cutoffs. The per-query CSV records every initial vector, classification, and explored-state count.

This oracle validates small instances of the code. It does not replace the mathematical all-initial-state theorem, and its conservative finite instances do not establish industrial performance on unbounded nets. The separate unbounded-output example exercises the infinite-state semantics without pretending that a finite forward oracle exhausted it.

### 11.3 Differential arithmetic and parameter tests

| Check family                                     | Count | Disagreements |
|--------------------------------------------------|-------|---------------|
| Complete forward-oracle queries                  | 3,920 | 0             |
| Exhaustive scalar predecessor identity checks    | 2,808 | 0             |
| Natural-pair versus signed-effect word summaries | 1,000 | 0             |
| Direct finite-word executions                    | 7,000 | 0             |
| Expanded small repetition executions             | 2,000 | 0             |
| Scalar-parameter diagnostic queries              | 2,278 | 0             |
| Large-parameter diagnostic queries               | 6,700 | 0             |
| Concrete paired-program observations             | 1,298 | 0             |

Table 2: Different units of validation. Rows overlap in their underlying generated inputs and must not be added as an independent-example total.

The scalar predecessor check ranges over  $c, p, b \in \{0, \dots, 5\}$  and  $m \in \{0, \dots, 12\}$ , comparing direct enabled-and-covering semantics with the threshold formula. The word suite generates 1,000 words of length at most twelve over small random nets. Seven starts per word compare direct firing against the producer’s summary. Two starts per word compare a small power, with repetition count at most eight, against concrete expansion. The checker also reconstructs the signed summary independently.

The random parameter block generates 134 scalar and 67 Pareto certificates; two additional named examples of each kind bring the stored totals to 136 and 69. The 67 random Pareto cases use 617 certified grid cells in total. Their 6,700 extra parameter observations deliberately include values beyond the certified caps. Those observations are diagnostics; Theorem 7.2, not extrapolation from the observations, justifies the global conclusion.

The separate equivalence suite constructs 120 program pairs. It finds 62 equal and 58 different endpoint partial functions, checking a concrete separator for each negative case and twenty random starts for each positive case. One additional symbolic pair compares  $10^{100}$  unit increments with a single increment by  $10^{100}$ . This yields the 63 positive equivalence records. The huge pair is checked by exact summaries, not expanded execution, and does not preserve primitive step count.

### 11.4 A controlled acceleration ablation

A separate block uses seed 41027 and the same original-net checker. On 24 small productive nets, plain and accelerated search return identical minimal frontiers. The aggregate candidate counts are 396 without acceleration and 171 with it. These are expansion counts, not wall-clock speedups.

Three larger cases use the same candidate budget, 100, and insertion budget, 200:

| Bad threshold | Plain mode     | Accelerated mode      | Receipt size |
|---------------|----------------|-----------------------|--------------|
| $10^4$        | UNKNOWN at 100 | Checked, 2 candidates | 3 run nodes  |
| $10^6$        | UNKNOWN at 100 | Checked, 2 candidates | 3 run nodes  |
| $10^{100}$    | UNKNOWN at 100 | Checked, 2 candidates | 3 run nodes  |

Each input is the add-one net, and each successful frontier is  $\{0\}$ . This is an actual comparison between two modes of this prototype. It does not measure a Lean tactic or an industrial model checker. The 27 acceleration-block certificates raise the stored frontier count from 67 to 94; the primary 67-case timing statistics below are not silently recomputed over this different workload.

### 11.5 Corruption controls and genuine unknowns

The unit tests include malformed vectors, duplicate JSON keys, unsupported transition fields, Boolean/integer confusion, invalid node references, false enabling claims, altered original queries, missing closure obligations, false parameter minima, and insufficient budgets. Eight separately stored deliberately invalid records test deletion of positive witnesses, missing target/transition obligations, changed net binding, an apparently nonnegative but disabled huge run, an empty infinite loop, and a false parameter minimum. All eight are rejected. These overlap mechanisms already tested in the unit suite; no random-mutant rejection percentage is asserted.

Four recorded controls return UNKNOWN: an unboundedness request on a stuttering loop, a growth request on a finite depletion path, a zero candidate budget, and an insufficient parameter box budget. These outcomes exercise the distinction between failing to find a witness and proving a negative statement. The test does not reward the worker for changing a claim until it becomes easy.

During implementation, a boundary review identified that ordinary Python structural equality permits a Boolean to compare equal to an integer. The final checker uses type-aware recursive identity, and two regression tests cover this issue. This was an engineering correction, not a discovered false Lean proof; no Lean checker existed in this environment.

### 11.6 Runtime observations without a tactic benchmark

For the 67 small frontier cases, the recorded median producer time is approximately 0.0443 milliseconds and the median checker time is 0.0609 milliseconds. The largest final basis has 36 vectors, the largest candidate count is 180, and the runs perform 39 dominance deletions in total. These are single-run local observations, retained in the CSV rather than promoted to a speedup claim. They mainly show that the delivered examples are small and that checking is not free.

The huge compressed run has three stored nodes and is checked without executing a single represented transition. Its recorded time is not a prediction of Lean-kernel time. Kernel reduction, proof-term size, array representation, and source transport remain to be measured.

### 11.7 What these results do not establish

No Lean source was compiled for this package, no source-level theorem was discharged, and no theorem axiom inventory was inspected. The source bridge, Boolean-checker soundness theorem, and tactic wrapper remain design work. Python checking is neither a formal proof of checker correctness nor protection against every malformed input.

No existing Forge suite was rerun here, and no head-to-head evaluation with **grind**, **Aesop**, **omega**, **SMPT**, or another solver was performed. Generated instances are not a representative sample of the user's developments or of a Petri-net competition. The contribution is a precisely

scoped implemented algorithmic lane with independent finite checks, mathematical proofs, and a concrete path to Lean, not a claim that Forge has already become a more successful prover.

## 12 A deployment path with falsifiable priorities

### 12.1 First deliver the finite checker theorem

The smallest useful end-to-end release is not a general Petri-net toolchain. It is an exact finite-multiset source adapter, the predecessor lemma, compressed run semantics, and the frontier checker theorem. Its acceptance example should quantify over an unbounded initial population and generate a reusable theorem. That is a concrete new proof capability rather than another worker whose result never reaches the kernel.

The first producer can be the supplied Python implementation. Its JSON records are already separated from its data structures and search code. After the Lean-side contract is proved, productive-transition acceleration costs only a producer option: the same finite checker accepts the stronger search's receipts. Parameter and equivalence checkers should be added only after one original source problem completes this path.

### 12.2 Make optimization preserve the mathematical contract

The current active-set implementation is intentionally simple. A one-dimensional basis can be represented by its least element. In two dimensions, a sorted skyline representation can accelerate dominance queries and deletion. In higher dimensions, prefix tries or range-search structures may help but require empirical evaluation; their purpose is to reduce producer work, never to weaken the final indexed comparisons.

A router can first inspect dimensions, support sparsity, and transition effects. Productive transitions qualify immediately for the proved power shortcut. A detected mixed-effect transfer with a large target warns of a potentially large antichain, as the output-size example shows. That signal should prefer a smaller scoped query or an existing arithmetic-invariant lane, not merely retry the same global frontier with a larger budget.

For a fixed initial state, one may stop early on an independently replayed positive witness. For a global frontier, early positive termination is invalid because other initial states are still unclassified. These should be different request kinds. Likewise, a formula-only affine-family result (16) should remain available when optional Pareto enumeration is refused. This would improve the current CLI's usability without changing its mathematics.

### 12.3 Evaluate where a general proof assistant actually benefits

A meaningful next benchmark should contain Lean source theorems in several independent families: finite-state process populations represented as multisets, resource-allocation protocols with unbounded counts, guarded counter-program transformations, and initialization synthesis with several resource parameters. Every goal should retain its original statement and exact source bridge.

Compare at least the base Forge pipeline, base plus this worker, and base plus the worker with power acceleration. Record translation time, candidate search time, certificate-check time, source-proof reconstruction time, final proof size, axiom dependencies, and all UNKNOWN outcomes. Reuse the same assumptions and theorem database across variants. A theorem that becomes easy after manually supplying the final frontier should be distinguished from one where the system discovered it automatically.

The failure categories should say what happened: unsupported source semantics; bridge construction failed; producer budget exhausted; certificate invalid; kernel reconstruction failed; or successfully replayed source theorem. Collapsing all of these into success/failure would hide the main engineering risks. This proposal does not report such a benchmark because it was not performed.

## 13 Conclusion

The extension proposed here changes the kind of answer Forge can aim to return. A successful resource analysis produces an exact unsafe initial-state precondition, positive executions at its minimal points, and parameter consequences valid beyond any tested population. Guard-sensitive summaries simultaneously support compressed witnesses, endpoint equivalence, and an exact repetition shortcut whose benefit is demonstrated within the delivered prototype.

The mathematical fragment is deliberately narrow: fixed-dimensional ordinary resource transitions, upward-closed bad patterns, and nonnegative affine initial families. Within it, the central frontier and parameter claims have complete proofs, and the finite receipt conditions avoid formalizing the untrusted search. Beyond it, the source bridge determines which conclusions survive abstraction. In particular, a safety simulation does not automatically authorize a counterexample, and a finite lasso does not automatically provide a fair or non-stuttering source execution.

The package’s strongest executed result is not a new prover success rate. It is a working set of exact algorithms, an independently implemented finite checker, complete small-instance oracles, corruption and refusal controls, and a producer improvement that turns a  $10^{100}$ -step backward descent into a three-node receipt under the unchanged checker. The next decisive result would be a source-level Lean theorem with the original quantifiers and guards intact. The implementation contract in this article makes that result concrete and testable.

## A Rebuild and run the package

The archive contains `article.tex`, `references.tex`, the compiled `article.pdf`, the Python source, named JSON examples, recorded certificates and diagnostics, and implementation/review notes. No search-library installation or network connection is needed to run the Python material.

From the archive’s root, run:

```
python -m unittest discover -s prototype/tests -v

python -S prototype/replay.py \
  results/frontiers.jsonl results/accelerated-frontiers.jsonl \
  results/thresholds.jsonl results/parameters.jsonl \
  results/runs.jsonl results/lassos.jsonl \
  results/equivalences.jsonl
```

The expected recorded run has 68 tests and 423 checked replay records. The replay reports `search_imported: false`. Timings can change; the semantic classifications should not.

To create a new receipt from a supplied example:

```
python -S prototype/solve.py examples/mutex.json \
--out my-results/mutex.json

python -S prototype/solve.py examples/broken_mutex.json \
--slope 1,0,0 --offset 0,1,0 \
--out my-results/threshold.json

python -S prototype/solve.py examples/mutex.json \
--family examples/population-permits.json \
--out my-results/parameters.json
```

Add `-accelerate` to enable the exact productive-transition shortcut. The solver refuses to overwrite an existing output, runs the independent checker before writing, and emits a one-line JSON replay record. Pass that file to `replay.py` for a second-process check. Exit status zero means Python checked; status one means invalid or unreadable input/certificate; status two means resource refusal or UNKNOWN.

To regenerate the experiment corpora into a fresh directory:

```
python -S prototype/experiments.py --out reproduced-results
python -S prototype/equivalence_experiments.py \
--out reproduced-results
python -S prototype/acceleration_experiments.py \
--out reproduced-results
```

The first experiment refuses an existing summary rather than overwriting recorded evidence. The additional suites write their separately named files in the selected directory. The **Makefile** provides `test`, `replay`, `reproduce`, and `article` targets.

For the article, run `pdflatex -interaction=nonstopmode -halt-on-error article.tex` three times. The document uses stock pdfLaTeX packages and an included bibliography, with no shell escape, external bibliography program, downloaded fonts, or externally generated figures.

## B An evidence map for maintainers

| Artifact                                       | What it establishes or records                                                  |
|------------------------------------------------|---------------------------------------------------------------------------------|
| <code>results/unit-tests.txt</code>            | Passing focused tests, including semantic and malformed-input controls          |
| <code>results/replay-log.txt</code>            | Fresh-process replay counts and absence of producer/model imports               |
| <code>results/summary.json</code>              | Main generator parameters, environment, counts, and timing observations         |
| <code>results/forward-oracle.csv</code>        | Every complete conservative forward-search query and state-count diagnostic     |
| <code>results/frontier-costs.csv</code>        | Search/check timings and frontier/worklist sizes for the original 67 cases      |
| <code>results/acceleration-summary.json</code> | Same-prototype acceleration ablation and large-threshold results                |
| <code>results/equivalence-summary.json</code>  | Paired-program classifications and concrete observation count                   |
| <code>results/mutation-results.json</code>     | Eight deliberately invalid record outcomes and diagnostic reasons               |
| <code>results/unknown-controls.json</code>     | Four cases that must remain inconclusive rather than acquire negative authority |

| Artifact                                    | What it establishes or records                                                 |
|---------------------------------------------|--------------------------------------------------------------------------------|
| <code>results/named-conclusions.json</code> | Exact mutex and broken-mutex frontiers and population conclusions              |
| <code>docs/REVIEW_SCOPE.md</code>           | Reviewed baseline, non-duplication scope, and limitations of repository search |
| <code>docs/LEAN_PORT.md</code>              | Proposed Lean module/proof obligations, explicitly not a completed port        |
| <code>docs/ACCELERATION.md</code>           | Implemented power shortcut, its restrictions, and the associated ablation      |

## C Boundary checklist

Before claiming a new source-level success, check the exact meaning of the object. Is the target upward coverability rather than exact equality? Are consumption guards retained? Is the number of place kinds fixed? Does the initialization have nonnegative coefficients? Is a global frontier being replaced by a fixed-initial-state invariant? Do positive abstract paths lift to source paths? Does an infinite source claim include a non-stuttering lift? Does program equivalence observe only endpoints and enabledness, or also traces and time?

For implementation integrity, check the exact object/query binding, Boolean-versus-integer distinction, vector dimensions, strictly earlier DAG references, target indices, and every original transition’s closure row. For performance reporting, distinguish represented steps from executed steps, checker cost from producer cost, small complete oracles from bounded tests, and Python acceptance from Lean kernel acceptance. These distinctions are the limits under which the algorithms in this article are correct.

## References

- [1] V. Reshetnikov and repository contributors. *Forge: merged design and assessment*. Revision 58ea206bd7ad501b930add568151217bcc7f82a2, inspected 15 September 2026. Repository overview and `article/sections/15-conclusion.tex`. <https://github.com/VladimirReshetnikov/Forge/tree/58ea206bd7ad501b930add568151217bcc7f82a2>.
- [2] Forge contributors. *Merged closure section*. Merged article, `article/sections/06-closure.tex`, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/article/sections/06-closure.tex>.
- [3] Forge contributors. *Quantitative certificates: probabilities, values, and adversaries*. Merged article, `article/sections/10-quantitative.tex`, same revision. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/article/sections/10-quantitative.tex>.
- [4] Forge contributors. *Ideas from Leant and Djex*. `docs/IDEAS-FROM-LEANT-DJEX.md`, inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/blob/main/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] V. Reshetnikov and repository contributors. *Leant—a Djex-based synthesis REPL for Lean 4*. Repository README, opening and current-scope material inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Leant>.
- [6] V. Reshetnikov and repository contributors. *Djex: type-directed expression synthesis*. Repository README, lines 1–160 inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Djex>.
- [7] T. Geffroy, J. Leroux, and G. Sutre. *Occam’s Razor Applied to the Petri Net Coverability Problem*. arXiv:1607.05956, 2016. In particular, the backward-set and finite-basis development in Sections 3–4 and its appendices. <https://arxiv.org/abs/1607.05956>. Published version DOI: <https://doi.org/10.1016/j.tcs.2018.04.014>.
- [8] Lean contributors. *The Lean Language Reference: The grind tactic*. Rolling official documentation, inspected 15 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [9] Mathlib contributors. *Mathlib.Data.Multiset.Count*. Rolling API documentation, inspected 15 September 2026; not compiled in this environment. [https://leanprover-community.github.io/mathlib4\\_docs/Mathlib/Data/Multiset/Count.html](https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Multiset/Count.html).
- [10] Mathlib contributors. *Mathlib.Data.Multiset.AddSub*. Rolling API documentation, inspected 15 September 2026; not compiled in this environment. [https://leanprover-community.github.io/mathlib4\\_docs/Mathlib/Data/Multiset/AddSub.html](https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Multiset/AddSub.html).
- [11] N. Amat and collaborators. *SMPT—Satisfiability Modulo Petri Nets*. Author’s project documentation, inspected 15 September 2026. <https://homepages.laas.fr/namat/project/smpt/>. The proposed integration in this article was not executed.